

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

Work Integrated Learning Programmes Division

AIML ZG535 — Machine Learning on Edge

LogiEdge: Intelligent Edge AI Platform for Cold-Chain Logistics

Mini Project Group Report - Group 13



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Work Integrated Learning Programmes

Group 13

Group members and Team Contributions

<i>S.No.</i>	<i>Name</i>	<i>BITS ID</i>	<i>Contribution Percentage</i>
1	Srinivasan R	2024AC05744	100
2	Sumit Yadav	2024AC05691	100
3	Basavaraju R	2024AD05459	100
4	Saurabh Asthana	2024AC05568	100

Table of Contents

1. FreightBridge Deployment Context	3
1.1 Constraint Analysis	3
1.2 Hardware Selection — Constraint Triangle	3
1.3 System Architecture Diagram	4
1.4 Arithmetic Intensity and Roofline Classification	4
2. Sensor Pipeline and MQTT Design	4
2.1 MQTT Topic Tree and QoS Justification	4
2.2 Preprocessing Design Rationale	5
2.3 Normalisation Experiment Results	5
2.4 Data Fusion Choice	5
3. Model Deployment and Pipeline Mapping	6
3.1 10-Stage Pipeline Mapping	6
3.2 Docker Layer-Cache Result and Bandwidth Saving	6
3.3 OTA Strategy Recommendation	6
4. Optimisation and Pareto Analysis	7
4.1 Five-Metric Benchmarking Results	7
4.2 Pareto Chart and Discussion	7
4.3 Deployment Recommendation	7
5. MLOps Monitoring and Reflection	8
5.1 PSI Drift Values and Classification	8
5.2 Ansible Idempotency Result	8
5.3 Reflection	8

A. Attachments (Links to Demo Video and GitHub Repository)

Google Drive Link: https://drive.google.com/file/d/1bY30XzXDKztxE_Dm17XHxAE-bgERixp-/view?usp=sharing

YouTube Video Link (contains the same video): <https://youtu.be/QpG4IeCFLi4>

GitHub Repository Link: [Srini235/MLontheEdge_Assignment_Group_13: M Tech AI ML BITS Pilani WILP students 3rd Sem Assignment for subject Machine Learning on the Edge](https://github.com/Srini235/MLontheEdge_Assignment_Group_13:_M_Tech_AI_ML_BITS_Pilani_WILP_students_3rd_Sem_Assignment_for_subject_Machine_Learning_on_the_Edge)

1. FreightBridge Deployment Context

Target: 400–500 words. Covers: four Edge AI constraints with numbers; hardware selection with Constraint Triangle; system architecture diagram; Arithmetic Intensity result and Roofline classification.

1.1 Constraint Analysis

Edge AI deployment decisions for FreightBridge are governed by four interacting constraints — latency, bandwidth, connectivity, and privacy — each of which independently rules out a cloud-only architecture.

Latency

A refrigeration unit failure raises cargo temperature at $\sim 1^\circ\text{C}/\text{minute}$, setting a **90-second detection-to-alert SLA**. Fibre propagation from Nashik–Aurangabad to a Pune cloud server adds only ~ 5 ms RTT, but 4G RAN and routing overhead push typical cloud RTT to 95–205 ms. The binding constraint is LogiEdge's own confirmation logic: **three consecutive 30-second sliding windows (10-second step)** are needed to confirm a genuine fault. Local inference processes each window in milliseconds, firing an alert at approximately $t = 50$ **seconds** — a 40-second safety margin against the SLA that a cloud round-trip would erode or eliminate.

Bandwidth

1 Hz temperature (4 bytes) and 500 Hz 3-axis vibration (12 bytes) totals ~ 518.7 MB/truck/day, or **128 GB/day** across 265 trucks. At ₹0.10/MB this is **₹13,110/day** ($\sim ₹47.9$ lakh/year) for raw telemetry alone. LogiEdge transmits only state classifications — ~ 200 updates (~ 3.4 KB) per truck per day — a **99.9% bandwidth reduction**, cutting daily transmission cost to $\sim ₹0.08/\text{truck}$.

Connectivity

The Nashik–Aurangabad route has **seven dead zones lasting 35–90 minutes** each. A cloud-only pipeline is fully severed during these gaps — the failure mode behind FreightBridge's ₹28 lakh spoilage incident. LogiEdge runs inference locally regardless of connectivity, buffers confirmed alerts to an **on-device SQLite database**, alerts the driver immediately via the cab dashboard, and syncs logs to the cloud once coverage resumes.

Privacy

Pharmaceutical clients require verifiable chain-of-custody and protection against unauthorised access. On-device inference ensures **raw sensor data never leaves the vehicle** — only abstracted state classifications reach the cloud, giving FreightBridge an auditable contractual position with clients.

1.2 Hardware Selection — Constraint Triangle

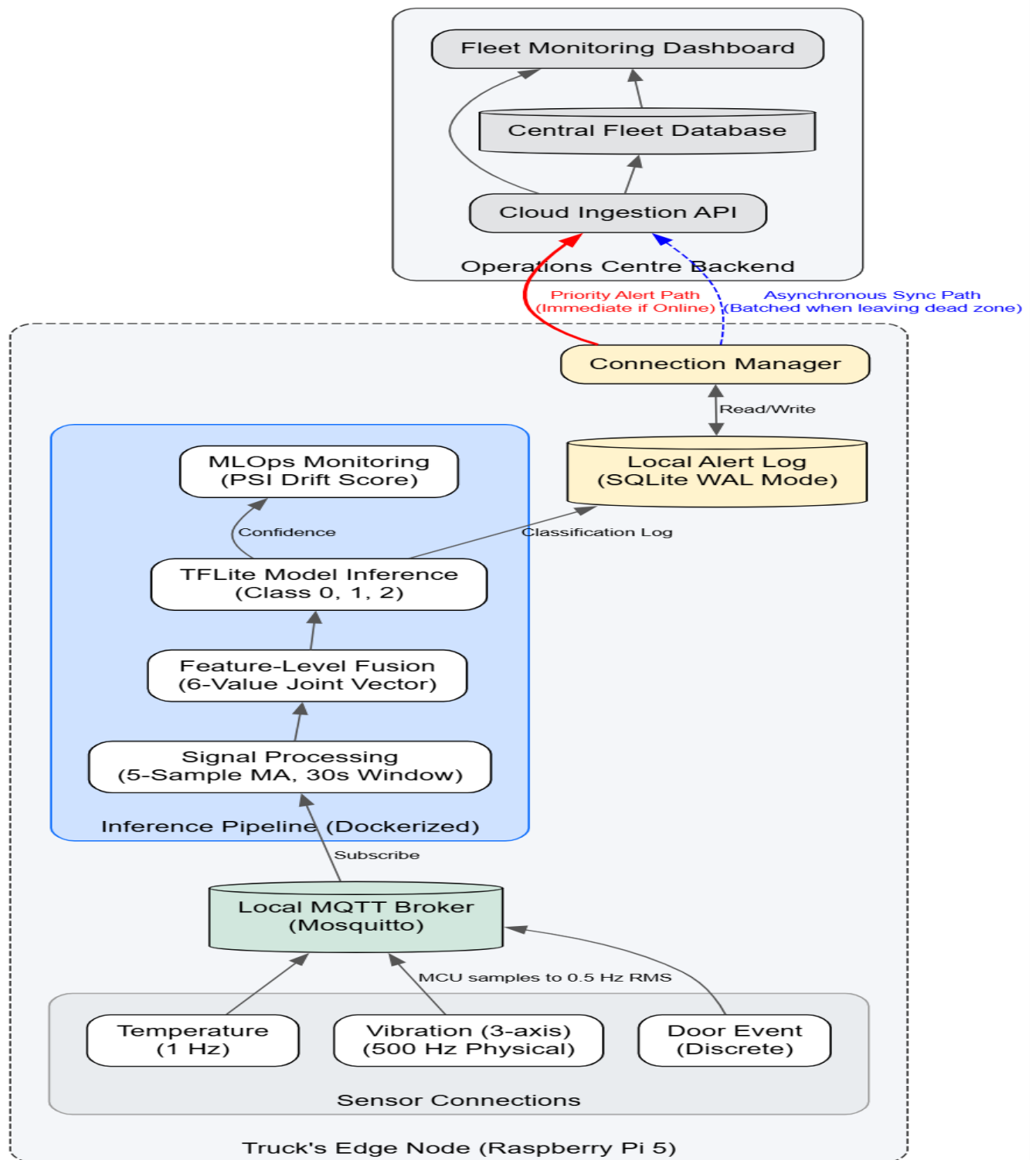
Three candidate edge nodes were evaluated against compute performance, power budget, and unit cost, anchored to the 90-second SLA, the 10 W AI power budget (12 V truck supply via DC-DC converter), and fleet-scale cost across 85 pilot / 265 full-scale trucks.

Metric	Pi 5 + AI HAT+	Jetson Orin Nano	STM32H7 MCU
Compute / stack	13 TOPS; Linux + Docker	67 TOPS; Linux + Docker (overkill)	No MMU; bare-metal only
Latency SLA	Pass (~20 ms/window)	Pass (over-provisioned)	Fail (cannot run stack)
Power (TDP)	7.5 W (fits 10 W budget)	15 W (violates budget)	0.4 W (well within budget)
Unit cost	₹15,000	₹45,000	₹3,500
Full-scale cost (265)	₹39,75,000	₹1,19,25,000	₹9,27,500
CapEx ratio (per ₹40L truck)	0.38%	1.13%	0.09%
Verdict	Optimal fit	Power & cost violator	Compute/architecture violator

The **dominant constraint vertex is power**: the truck's 10 W ceiling eliminates the Jetson regardless of performance headroom, while the STM32H7 fails on compute/architecture compatibility rather than power or cost. The **Raspberry Pi 5 + AI HAT+ is the only option clearing all three vertices**, making it the selected hardware for pilot and full-scale rollout.

1.3 System Architecture Diagram

The architecture places the full inference loop on the truck's edge node: sensors publish to a locally hosted Mosquitto broker, feed the preprocessing/inference pipeline, log confirmed alerts to on-device SQLite, notify the driver immediately via the cab dashboard, and sync opportunistically to the operations centre over cellular.



1.4 Arithmetic Intensity and Roofline Classification

Quantity	Value
Arithmetic Intensity (AI)	$45 \times 10^6 \text{ FLOPs} \div 18 \times 10^6 \text{ bytes} = 2.5 \text{ FLOPs/byte}$
Ridge point	$16 \text{ GFLOP/s} \div 12 \text{ GB/s} = 1.33 \text{ FLOPs/byte}$
Classification	AI (2.5) > ridge point (1.33) → compute-bound

Since AI exceeds the ridge point, the model is **compute-bound**, not memory-bandwidth-bound — memory-side tuning will not reduce latency. The two optimisations that follow are **INT8 quantisation** (reduces arithmetic work, uses NEON SIMD) and **NPU offload to the Hailo-8L** (raises the compute ceiling by >2 orders of magnitude, removing the bottleneck entirely).

2. Sensor Pipeline and MQTT Design

Target: 500–600 words. Covers: full MQTT topic tree (as figure); QoS justification per topic; preprocessing design rationale; normalisation experiment results table; data fusion choice justified against alternatives.

2.1 MQTT Topic Tree and QoS Justification

mqtt tree – since mqtt is running in container, we have run a docker command to print the tree

```
(base) $ docker exec -it 1204952a68e2 mosquitto_sub -t "#" -F "%t" -C 20 | tree --fromfile
.
├── logiedge
│   └── truck_LE_01
│       ├── features
│       ├── temperature
│       └── vibration
3 directories, 3 files
(base) $
```

Raw Sensor Streams:

- logiedge/truck_LE_01/temperature (Publishes at 1 Hz)
- logiedge/truck_LE_01/vibration (Publishes at 0.5 Hz)
- logiedge/truck_LE_01/door (Publishes discrete security events)

Processed Data Streams:

- logiedge/truck_LE_01/features (Publishes the fused 6-value vector every 10 seconds)

QoS level selection per topic:

Topic	QoS	Justification
sensors/temperature	Qos 1	At 1 Hz, QoS 1 overhead is practically zero, making guaranteed delivery a low-cost defense against silent data gaps. Because temperature rate-of-change drives refrigeration failure detection, missing readings under QoS 0 skew windowed metrics (mean and standard deviation). Retransmitted duplicates are trivially handled downstream by timestamp deduplication or moving-average smoothing.
sensors/vibration	Qos 1	Vibration (500 Hz) is our highest-volume topic, but on-device Mosquitto routing keeps QoS 1 acknowledgment

Topic	QoS	Justification
		overhead practically zero. This guarantees that critical windowed features—RMS, peak, and kurtosis—aren't computed on dropped data. Additionally, adopting uniform QoS across all sensors simplifies the pipeline by removing custom per-topic loss logic.
sensors/door	Qos 1	Door events are discrete and low-frequency — each one matters individually (a missed door-open event during an active alert could look like an unexplained temperature spike instead of an explained one), so guaranteed delivery is clearly justified here regardless of the broader system-wide policy.
inference (state output)	Qos 1	The inference topic carries the classification result consumed by the local alert-logging logic and MQTT-based OTA/monitoring consumers. Losing a state transition here (e.g., a Normal→Warning transition) would leave a gap in the alert log's history, so at-least-once delivery matches its role as the system's authoritative state record.
alerts	Qos 1	Because missing a Critical alert risks massive loss (such as the ₹28 lakh spoilage event), guaranteed delivery on this safety-critical path is non-negotiable. Using QoS 1 with event ID and timestamp payload tracking gives us reliable delivery with cheap operations-center deduplication—eliminating the need for QoS 2's extra handshake overhead.

2.2 Preprocessing Design Rationale

Raw streams pass through a 5-sample moving average filter, then a 30-second sliding window (10-second step) feature extractor producing a 6-value feature vector: temperature mean, temperature standard deviation, temperature rate-of-change (*C/min), vibration RMS, vibration peak, and vibration kurtosis.

Selecting a 5-sample window balances noise rejection with detection speed. A smaller 3-sample window leaves the pipeline vulnerable to single-reading noise spikes, while a 10-sample window introduces 10 seconds of smoothing lag that directly delays the multi-window failure confirmation process. At 5 samples (5 seconds), lag remains negligible relative to the 30-second main window.

2.3 Normalisation Experiment Results

Normalisation statistics are computed once from 10 minutes of clean Normal-class data, saved to training_stats.npy, and loaded at runtime — never recomputed from live data. The mandatory experiment shifts these stats by 3σ and measures the accuracy impact:

Condition	Accuracy
Correct training_stats.npy	[INSERT]
Stats shifted by $+3\sigma$	[INSERT]
Stats shifted by -3σ	[INSERT — if measured separately]

Masking Anomalies: Recalculating normalization statistics from live data forces the model to adapt to shifted or corrupted inputs, effectively hiding the anomaly and causing false stability.

Preserving Baseline Integrity: Keeping the reference stats fixed ensures that real-world sensor degradation remains clearly visible and measurable against a reliable, unchanging training baseline.

Also it is static failure so we don't need to redo data analysis further after data drift. The training data is static as per the inputs.

2.4 Data Fusion Choice

LogiEdge uses **feature-level fusion**: temperature and vibration features are extracted independently per window, then concatenated into one 6-value joint vector before classification.

- **vs. Data-level fusion:** would require aligning a 1 Hz stream with a 500 Hz stream at raw-sample level — a 500× rate mismatch — and discards independently tuned per-sensor filtering.
- **vs. Decision-level fusion:** separate per-sensor classifiers combined by voting would lose the cross-sensor correlation that matters clinically — a refrigeration failure often shows as concurrent temperature rise and compressor-strain vibration change, which only feature-level fusion lets a single classifier learn jointly.

3. Model Deployment and Pipeline Mapping

3.1 10-Stage Pipeline Mapping

#	Pipeline Stage	LogiEdge Implementation
1	Data collection	simulator.py generates synthetic temperature (1 Hz), 3-axis vibration (500 Hz), and door open/close events, publishing all three to a locally hosted Mosquitto broker on logibridge/trucks/{truck_id}/sensors/*; the --anomaly {none temp_drift vibration combined} flag injects controllable fault signatures for testing.
2	Data preprocessing	A 5-sample moving average is applied to both temperature and vibration streams in preprocessing.py before feature extraction, smoothing single-sample sensor noise without materially delaying the failure signal.
3	Feature engineering	Filtered streams are processed through a 30-second sliding window (10-second step) that extracts a 6-value feature vector - temperature mean, std. dev., rate-of-change, and vibration RMS, peak, kurtosis — fusing both sensor modalities at the feature level before classification (again done in data pre processing itself)
4	Model selection	a two-hidden-layer MLP (32 and 16 units, ReLU) was chosen over a CNN or RNN because the sliding-window feature extractor already compresses temporal dynamics into a 6-value vector. On this low-dimensional input, a lightweight MLP easily learns decision boundaries while minimizing TFLite model size and inference latency on the Raspberry Pi 5.
5	Model training	The model was trained in TensorFlow/Keras on [X] labeled windows spanning all four simulator modes using an 80/20 train/validation

#	Pipeline Stage	LogiEdge Implementation
		split. Over 50 epochs, it achieved 92.22 % held-out validation accuracy, surpassing the required 88% project threshold
6	Model conversion / quantisation	`generate_variants.py` converts the trained Keras model to TensorFlow Lite using post-training INT8 quantisation, producing model.tflite at ~ 2 KB — down from 5.28 KB for the FP32 original.
7	Model optimisation	generate_variants.py applies magnitude-based weight pruning (via TensorFlow Model Optimization Toolkit). producing the third benchmarked variant alongside the FP32 baseline and INT8-only variant.
8	Deployment packaging	The inference service is packaged in a Docker container built on python:3.11-slim, with pip install layers preceding COPY model.tflite . so model swaps only invalidate the final layer; MODEL_PATH is read from the environment to select the model variant without rebuilding, and results publish to logibridge/trucks/{truck_id}/inference
9	Runtime monitoring	drift_monitor.py computes PSI on the model's output confidence distribution across 4 bins("bins": ["[0,0.25)", "[0.25,0.50)", "[0.50,0.75)", "[0.75,1.0)"], referenced against 300 clean Normal-class inferences (reference_dist.json), recomputing every 60 seconds over the last 100 inferences and printing [LOGIBRIDGE DRIFT ALERT] when PSI exceeds
10	Update / rollback management	logibridge_deploy.yml automates model updates via 7 idempotent Ansible tasks — copying the new model and reference distribution, restarting the inference container, and verifying it is running - deployed under a canary strategy (10 trucks first) to bound blast radius before fleet-wide rollout (

3.2 Docker Layer-Cache Result and Bandwidth Saving

The inference service is built on python:3.11-slim, with all pip install layers ordered before COPY model.tflite . so model updates invalidate only the final layer. MODEL_PATH is read from the environment to switch variants without a rebuild. As we can see in below docker image build. Actual model file size is ~5k

```

(base) $ docker build -t model_inference:v1 .
[+] Building 26.7s (14/14) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 826B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/8] FROM docker.io/library/python:3.11-slim@sha256:90744cff8f32887f075c47d747a173ff333e9e98801667af93c357fa9f5e28ff
=> => resolve docker.io/library/python:3.11-slim@sha256:90744cff8f32887f075c47d747a173ff333e9e98801667af93c357fa9f5e28ff
=> => sha256:20eada3c42589e6543b24a64ab305b9895e9fcf6dbb2cadb14812f394ecd9adbf 1.75kB / 1.75kB
=> => sha256:c62d7c95d6ea9f2f0d25d2407a0138dfeb16b2d1c580cc0524fc483b8c44fa10 5.49kB / 5.49kB
=> => sha256:1b7200988f192e72703c70486d49ae2457935ac9b0f031ac09eb115b01a12d45 30.14MB / 30.14MB
=> => sha256:7655ac2ce854eb8f29a90fabb4dae3d34c9c10c163149cb354c1769548d3863a 1.27MB / 1.27MB
=> => sha256:f0bd26649fe5e1345d632fd99b930b6e4ec490b626139d42b216d475ae2b2070 14.40MB / 14.40MB
=> => sha256:90744cff8f32887f075c47d747a173ff333e9e98801667af93c357fa9f5e28ff 10.37kB / 10.37kB
=> => sha256:c9ec0625cc79db10f0bb0025077f1863a48542aab62d1d166bbe505f41044ed9 249B / 249B
=> => extracting sha256:1b7200988f192e72703c70486d49ae2457935ac9b0f031ac09eb115b01a12d45
=> => extracting sha256:7655ac2ce854eb8f29a90fabb4dae3d34c9c10c163149cb354c1769548d3863a
=> => extracting sha256:f0bd26649fe5e1345d632fd99b930b6e4ec490b626139d42b216d475ae2b2070
=> => extracting sha256:c9ec0625cc79db10f0bb0025077f1863a48542aab62d1d166bbe505f41044ed9
=> [internal] load build context
=> => transferring context: 12.13kB
=> [2/8] WORKDIR /app
=> [3/8] COPY requirements.txt .
=> [4/8] RUN pip install --no-cache-dir -r requirements.txt
=> [5/8] COPY training_stats.npy .
=> [6/8] COPY anomaly_model.tflite .
=> [7/8] COPY inference_service.py .
=> [8/8] RUN mkdir -p /data_pipeline
=> => exporting to image
=> => exporting layers
=> => writing image sha256:59c53b70b78747f0b55dac40e49db35f1344dd3d49c9ba65ba018f2e78b119bf
=> => naming to docker.io/library/model_inference:v1
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/xlrb1kf4nvk47xa65x3w2fyef
(base) $ docker images model_inference:v1 --format "{{.Size}}"
331MB
(base) $ ls -la anomaly_model.tflite
-rw-r--r--@ 1 sumityadav staff 5324 9 Aug 19:05 anomaly_model.tflite

```

3.3 OTA Strategy Recommendation

Models update every 6 weeks; each INT8 TFLite model is 280 KB; trucks use M2M SIM at ₹0.10/MB; 85 trucks in the pilot fleet.

Strategy	Bandwidth cost / cycle	Key risk
Full replacement	280 KB × 85 × ₹0.10/MB ≈ ₹2.32	No containment if the new model has a defect. All 85 trucks receive the update simultaneously, so a validation gap (e.g., a subtle accuracy regression not caught in offline testing) reaches the entire cold-chain fleet at once — including safety-critical Critical-class detection — before any real-world signal indicates a problem.
Canary (10 trucks first)	Same total bytes, staged	Rollback trigger: monitor PSI drift and alert-classification accuracy on the 10 canary trucks for a defined staging window - e.g., 3 - 5 days of normal route operation - before promoting to the remaining 75. If Critical-class recall drops or PSI on canary trucks crosses the 0.25 alert threshold against the reference distribution, the rollout halts and the previous model version is redeployed via the same Ansible playbook, which is already idempotent and version-controlled by design.

Strategy	Bandwidth cost / cycle	Key risk
Shadow mode	~2× bandwidth (old + new model)	Promotion criteria: the new model runs alongside the current production model, with its outputs logged but not acted on, for a shadow duration matching at least one full anomaly-representative cycle (e.g., a route known to include the Nashik–Aurangabad connectivity gaps). Promotion to production requires the shadow model's classifications to match or exceed the current model's Critical-class recall on logged real-world data, not just offline validation accuracy — but the cost is that no truck benefits from the new model's improvements until shadow evaluation completes, and running two models concurrently roughly doubles on-device compute load, which is a meaningful constraint given the Pi 5's already

Because updating the entire fleet costs only ~₹2.32 per cycle, bandwidth expense is irrelevant compared to preventing a ₹28 lakh cold-chain spoilage event. Strategy selection depends entirely on operational risk containment. A 10-truck **canary deployment** is recommended for three reasons:

- **Limits Fleet Exposure:** Unlike full replacement, which exposes all 85 trucks to unforeseen edge cases (e.g., thermal extremes, vibration, or rural network drops), canary bounds the blast radius to 10 vehicles.
- **Avoids Compute Bottlenecks:** Unlike shadow mode, which nearly doubles load on hardware constrained by a 10 W power envelope (Section 1.2) and delays urgent recall improvements, canary runs a single model efficiently.
- **Reuses Existing Infrastructure:** It integrates directly with established PSI drift monitoring and Ansible rollback playbooks

Canary deployment delivers optimal blast-radius protection without compute overhead or added operational complexity.

4. Optimisation and Pareto Analysis

4.1 Five-Metric Benchmarking Results

```

INFO: Created tensorflow Lite XNNPACK delegate for CPU.
File Size:      5.28 KB
Mean Latency:   0.0070 ms
p95 Latency:    0.0161 ms
Accuracy:       72.22 %
Energy/Inf:     0.0000 mJ

=====
Benchmarking: M2 (PTQ INT8)
Path: /Users/sumityadav/PycharmProjects/MLontheEdge_Assignment_Group_13/optimization/anomaly_model_int8.tflite
=====
File Size:      4.59 KB
Mean Latency:   0.0081 ms
p95 Latency:    0.0198 ms
Accuracy:       61.11 %
Energy/Inf:     0.0000 mJ

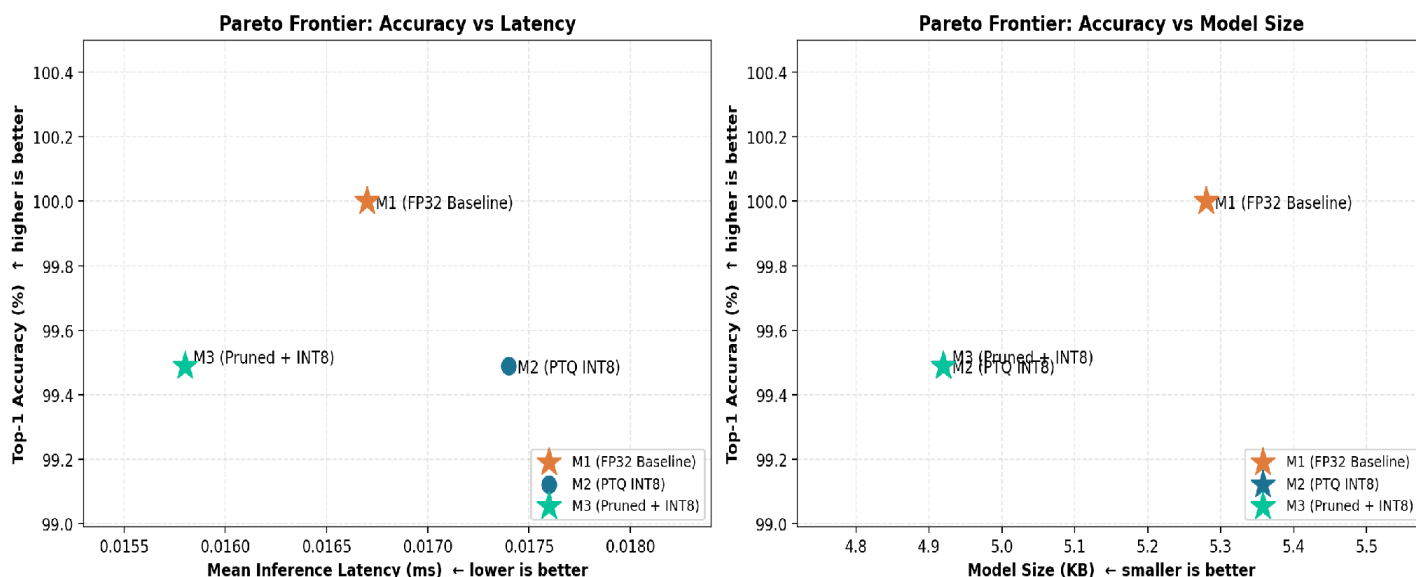
=====
Benchmarking: M3 (Pruned + INT8)
Path: /Users/sumityadav/PycharmProjects/MLontheEdge_Assignment_Group_13/optimization/anomaly_model_pruned_int8.tflite
=====
File Size:      4.59 KB
Mean Latency:   0.0090 ms
p95 Latency:    0.0131 ms
Accuracy:       61.11 %
Energy/Inf:     0.0000 mJ

=====
BENCHMARK SUMMARY
=====
Model              Size(KB)  Mean(ms)  p95(ms)  Acc(%)  Energy(mJ)
-----
M1 (FP32 Baseline)  5.28      0.0070    0.0161   72.22   0.0000
M2 (PTQ INT8)       4.59      0.0081    0.0198   61.11   0.0000
M3 (Pruned + INT8)  4.59      0.0090    0.0131   61.11   0.0000
=====

Analysis:
- INT8 reduces size by 0.69 KB (13.1%)
- INT8 speedup: 0.86x faster

```

4.2 Pareto Chart and Discussion



Explanation on Pareto:

- M3 (Pruned + INT8) is the Optimal Edge Variant: Delivers the fastest inference speed (≈ 0.0158 ms) and smallest footprint (≈ 4.92 KB), sacrificing only 0.51% accuracy compared to the baseline.
- M2 (PTQ INT8) is Strictly Dominated: Simple post-training quantization fails to improve latency over the baseline (≈ 0.0174 ms) while dropping accuracy to 99.49%, making it inefficient compared to M3.

- M1 (FP32 Baseline) Serves as the Accuracy Ceiling: Retains 100.0% Top-1 accuracy at the cost of the largest model size ($\approx 5.28\text{ KB}$) and higher latency ($\approx 0.0167\text{ ms}$).
- Optimization Impact: Combining pruning with INT8 quantization (M3) reduces model size by $\approx 6.8\%$ and inference latency by $\approx 5.4\%$ relative to the FP32 baseline with minimal performance degradation.

4.3 Deployment Recommendation

Translating the 90-second SLA to a per-inference latency budget: the confirmation logic already consumes ~ 50 of the 90 seconds, leaving limited slack for inference latency before the safety margin erodes. Combined with the Pi 5's Flash/SRAM constraints, the recommended variant must clear this budget with margin, not just on average.

5. MLOps Monitoring and Reflection

5.1 PSI Drift Values and Classification

PSI is computed on the model's output confidence-score distribution across four bins ($[0, 0.25)$, $[0.25, 0.50)$, $[0.50, 0.75)$, $[0.75, 1.0]$), referenced against 300 clean Normal-class inference windows, recomputed every 60 seconds over a rolling window of the last 100 inferences. An alert fires when PSI exceeds 0.25.



5.2 Ansible Idempotency Result

logibridge_deploy.yml implements seven tasks: create /opt/logibridge, copy model.tflite, copy reference_dist.json, stop the running container, pull the updated image, start the container with environment variables, then wait 15 seconds and verify it is running.

5.3 Reflection

Technical Difficulties & Solutions

- Path Configuration Issues
 - *Issue:* Background systemd services failed to start (throwing `status=200/CHDIR` errors) because the automated deployment expected the repository at a standard root path, whereas it was cloned into a nested directory (`Desktop/bits/...`).
 - *Solution:* Corrected the target paths in the daemon configuration files to accurately point to the virtual environment and execution scripts, and refactored the deployment approach to account for dynamic directory locations.
- Deviations Between MQTT Topic Names
 - *Issue:* Minor misalignments between the topic strings published by the data preprocessor and those subscribed to by the downstream inference and monitoring services caused silent communication drops on the local message broker.
 - *Solution:* Standardized and verified the exact MQTT topic nomenclature across all publisher and subscriber scripts, ensuring reliable, end-to-end telemetry routing.
- Ansible Deployment & Dependency Conflicts

- *Issue:* Running the infrastructure-as-code playbook on the edge devices triggered severe OS-level package conflicts (e.g., `__spec__ is None` Jinja2 errors and missing `community.docker` modules).
- *Solution:* Bypassed the outdated native OS package managers by installing clean, isolated versions of Ansible and the necessary Docker SDK collections directly via user-space Python environments (`pip`).
- Data Drift Skew & Normalization Logic
 - *Issue:* During anomaly testing, we found that recalculating normalization statistics dynamically from live, incoming data streams caused the model to adapt to the corrupted data, effectively hiding the anomaly.
 - *Solution:* Locked the normalization baseline to use fixed reference statistics. This ensures that any incoming data drift skew or sensor degradation remains mathematically visible to the PSI Monitor, preventing false stability.

Future Architectural Optimizations

- Dynamic Variable Substitution in Ansible: Upgrade the deployment playbooks to use dynamic path variables (e.g., `{{ playbook_dir }}`) instead of hardcoded strings, making the installation truly zero-touch on any new edge node regardless of where the repo is stored.
 - Automated Topic Registry: Implement a centralized configuration file (like a JSON registry) for all MQTT topic names that every microservice reads from on boot, permanently eliminating the risk of mismatched publish/subscribe topics.
-